Why, Which, and How of Algorithmic Techniques

Prefix Sum

Why Use Prefix Sum?

Efficiency: Prefix sums allow you to calculate the sum of elements in subarrays in constant time by preprocessing the data.

Use Case: Useful when you need to calculate the sum of elements in subarrays multiple times, optimizing time complexity.

Which Problems Can Be Solved Using Prefix Sum?

- **Range sum queries**: Finding the sum of elements between two indices multiple times.
- **Subarray problems**: Finding subarray sums and other prefix-based optimizations.
- **Contiguous subarray conditions**: Problems involving conditions on subarrays or difference arrays.

How to Implement Prefix Sum?

Step 1: Build the Prefix Sum Array
You accumulate the sums of the elements in the array to form a prefix sum array. Here's how you do it in Java:

```
public static int[] buildPrefixSum(int[] arr) {
   int[] prefix = new int[arr.length];
   prefix[0] = arr[0];  // Initialize the first element of prefix array
   for (int i = 1; i < arr.length; i++) {
      prefix[i] = prefix[i - 1] + arr[i];  // Accumulate the sum for each prefix element
   }
   return prefix;
}
```

Step 2: Use the Prefix Array for Range Sum Queries
To calculate the sum of elements between two indices i and j, use the prefix sum array as shown below:

```
public static int rangeSum(int[] prefix, int i, int j) {
   if (i == 0) {
      return prefix[j];  // If i is 0, return the prefix sum at j
   } else {
      return prefix[j] - prefix[i - 1];  // Return the difference for range sum
   }
}
```

Carry Forward Approach

Why Use Carry Forward Approach?

Efficiency: The carry forward approach is used to maintain some form of running value from previous iterations, reducing redundant calculations and optimizing space and time complexity.

Use Case: It's useful in problems where decisions are made based on previous computations or elements, such as calculating cumulative results like sums or maximum values as the iteration progresses.

Which Problems Can Be Solved Using Carry Forward Approach?

- Finding maximum or minimum values in sequences where every element depends on its previous values.
- Counting or calculating cumulative values in arrays, strings, or sequences.

How to Implement Carry Forward Approach?

In this technique, a value is carried forward from one iteration to the next. For example, to find the maximum subarray sum in a simple approach, you carry forward the current sum and update it step by step.

Sliding Window Technique

Why Use a Sliding Window?

Efficiency: The sliding window technique optimizes time complexity by reusing computations for overlapping segments of an array or list.

Use Case: It's beneficial for problems requiring analysis of continuous subarrays, substrings, or **elements in a fixed range**.

Which Problems Can Be Solved Using a Sliding Window?

- Maximum or minimum subarray problems.
- String problems like finding substrings with specific properties.
- Subarray problems involving sum or product calculations.

How to Implement Sliding Window?

```
public static int maxSumSubarray(int[] arr, int k) {
   int windowSum = 0;
   for (int i = 0; i < k; i++) {
      windowSum += arr[i];
   }

   int maxSum = windowSum;
   for (int i = k; i < arr.length; i++) {
      windowSum += arr[i] - arr[i - k];
      maxSum = Math.max(maxSum, windowSum);
   }
   return maxSum;
}
```

Kadane's Algorithm

Why Use Kadane's Algorithm?

Efficiency: Kadane's algorithm is an efficient approach to solve the maximum subarray sum problem in linear time, eliminating the need for nested loops.

Use Case: It is ideal for finding the largest sum of contiguous subarrays, which is a common problem in coding competitions and interview questions.

Which Problems Can Be Solved Using Kadane's Algorithm?

- Maximum subarray sum problems.
- Dynamic programming problems involving optimization over a range.

How to Implement Kadane's Algorithm?

```
public static int maxSubArraySum(int[] arr) {
   int maxSoFar = arr[0], currMax = arr[0];
   for (int i = 1; i < arr.length; i++) {
      currMax = Math.max(arr[i], currMax + arr[i]);
      maxSoFar = Math.max(maxSoFar, currMax);
   }
   return maxSoFar;
}
```

Moore's Voting Algorithm

Why Use Moore's Voting Algorithm?

Efficiency: Moore's Voting Algorithm finds the majority element in linear time and constant space, making it highly efficient when searching for the majority element in arrays.

Use Case: It is ideal when the majority element (appearing more than n/2 times) needs to be found in a list.

Which Problems Can Be Solved Using Moore's Voting Algorithm?

- Finding the majority element in an array.
- Handling cases where elements appear frequently in sequences.

How to Implement Moore's Voting Algorithm?

```
public static int findMajorityElement(int[] arr) {
   int candidate = 0, count = 0;

   // Step 1: Find a candidate
   for (int num : arr) {
      if (count == 0) {
         candidate = num;
      }
      count += (num == candidate) ? 1 : -1;
```

```
    }

    // Step 2: Verify if the candidate is a majority
    count = 0;
    for (int num : arr) {
        if (num == candidate) {
            count++;
        }
    }
    return count > arr.length / 2 ? candidate : -1;
}
```

Hashmap

Why Use Hashmap?

Efficiency: Hashmaps provide constant time complexity for search, insert, and delete operations, making them highly efficient for managing key-value pairs.

Use Case: Useful when fast lookups and updates are required, such as frequency counting, caching, and mapping keys to values.

Which Problems Can Be Solved Using Hashmap?

- Counting frequency of elements.
- Caching frequently accessed data.
- Efficient lookup of associated values for given keys.

How to Implement Hashmap?

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {

        HashMap<String, Integer> map = new HashMap<>();

        // Insert key-value pairs
        map.put("apple", 2);
        map.put("banana", 3);

        // Accessing values
```

```
        int value = map.get("apple");

        // Check if a key exists
        if (map.containsKey("banana")) {
            System.out.println("Banana is present with quantity: " + map.get("banana"));
        }

        // Removing a key-value pair
        map.remove("apple");
    }
}
```

Sorting Techniques

Why Use Sorting?

Efficiency: Sorting organizes data, which can improve performance for tasks like searching, merging, or optimization problems that rely on ordering.

Use Case: Useful for problems that involve comparisons, searching in sorted arrays, or when ordered data is required.

Which Problems Can Be Solved Using Sorting?

- Searching in ordered datasets.
- Optimizing problems with sorted input.
- Merging sorted arrays, organizing ranked data.

How to Implement QuickSort Algorithm?

```
// Example: QuickSort implementation in Java
public class QuickSort {
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);

            // Recursively sort elements before and after partition
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }
```

```
  private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++) {
      if (arr[j] < pivot) {
        i++;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
      }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
  }
}
```

Binary Search

Why Use Binary Search?

Efficiency: Binary search reduces the search space by half in each iteration, achieving logarithmic time complexity, making it significantly faster than linear search for sorted datasets.

Use Case: Ideal when working with sorted arrays or lists and when the goal is to find the position of a target element.

Which Problems Can Be Solved Using Binary Search?

- Finding a target value in a sorted array.
- Optimization problems where solutions lie in a sorted or search space that can be halved.

How to Implement Binary Search?

```
public class BinarySearch {
  public static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;

    while (low <= high) {
      int mid = low + (high - low) / 2;
```

```
        // Check if target is present at mid
        if (arr[mid] == target) {
            return mid;
        }

        // If target is greater, ignore the left half
        if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // Return -1 if target is not present
    return -1;
}
```

Stack

Why Use a Stack?

LIFO Structure: Stacks follow a Last In, First Out (LIFO) order, which makes them ideal for problems where the most recent element needs to be accessed first.
Use Case: Useful in scenarios like parsing expressions (e.g., parentheses balancing), undo functionality in applications, and depth-first search (DFS) in graphs.

Which Problems Can Be Solved Using a Stack?

- Balanced Parentheses: Checking if an expression has balanced parentheses, brackets, or braces.
- DFS: Used for traversing graphs or trees in a depth-first manner.
- Function Call Management: Stacks help manage function calls during recursion in programming.

How to Implement a Stack?

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
```

```
    Stack<Integer> stack = new Stack<>();

    // Push elements
    stack.push(10);
    stack.push(20);

    // Pop element
    System.out.println("Popped: " + stack.pop());

    // Peek at the top element
    System.out.println("Top element: " + stack.peek());

    // Check if stack is empty
    System.out.println("Is stack empty? " + stack.isEmpty());
  }
}
```

Queue

Why Use a Queue?

FIFO Structure: Queues follow a First In, First Out (FIFO) order, making them ideal for tasks that require processing elements in the order they arrive.
Use Case: Queues are used in scenarios like scheduling tasks, handling requests in web servers, and breadth-first search (BFS) in graphs.

Which Problems Can Be Solved Using a Queue?

- BFS: Traversing graphs level by level using breadth-first search.
- Task Scheduling: Managing tasks that need to be processed in order, like printer queues.
- Round-Robin Algorithms: Used in CPU scheduling to fairly distribute processing time among tasks.

How to Implement a Queue?

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
  public static void main(String[] args) {
```

```
        Queue<Integer> queue = new LinkedList<>();

        // Enqueue elements
        queue.add(10);
        queue.add(20);

        // Dequeue element
        System.out.println("Dequeued: " + queue.poll());

        // Peek at the front element
        System.out.println("Front element: " + queue.peek());

        // Check if queue is empty
        System.out.println("Is queue empty? " + queue.isEmpty());
    }
}
```

Trees

a) Level Order Traversal

Why Use Level Order Traversal?
Processes each level of the tree in a breadth-first manner, which is useful for problems where you need to access elements level by level.

Which Problems Can Be Solved Using Level Order Traversal?
- Organizational charts: Reading each level of hierarchy from top to bottom.
- Breadth-first Search: Traversing graphs or trees level by level.

How to Implement Level Order Traversal?

```
public class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int x) { val = x; }
}

public void levelOrder(TreeNode root) {
    if (root == null) return;
```

```
    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);

    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.val + " ");

        if (node.left != null) queue.add(node.left);
        if (node.right != null) queue.add(node.right);
    }
}
```

b) Node to Root Path & Lowest Common Ancestor (LCA)

- Node to Root Path: This helps in solving problems where you need to trace the ancestry of a node.
- LCA: LCA allows you to find the lowest common ancestor of two nodes, which is useful in various tree problems.

Which Problems Can Be Solved Using Node to Root Path & LCA?
- **Tree navigation**: Finding the path from a node to the root.
- LCA: Solving problems that involve finding a common parent of two nodes in a tree.

How to Implement LCA?

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root;

    return left != null ? left : right;
}
```

c) Search in Binary Search Tree (BST)

Why Search in a BST?

- Efficient Search: A binary search tree allows fast search, insertion, and deletion in logarithmic time.

Which Problems Can Be Solved Using Search in BST?

- Fast lookups: Finding elements efficiently in a sorted manner.

- Data Storage: Managing and retrieving data in a dynamic set where elements can be inserted or deleted.

How to Implement Search in BST?

```
public TreeNode searchBST(TreeNode root, int val) {
   if (root == null || root.val == val) return root;

   if (val < root.val) {
      return searchBST(root.left, val);
   } else {
      return searchBST(root.right, val);
   }
}
```

d) Insert and Delete in Binary Search Tree (BST)

- Maintains Sorted Order: Binary search trees keep elements in a sorted manner, allowing for dynamic insertion and deletion while maintaining order.

Which Problems Can Be Solved Using Insert/Delete in BST?

- Dynamic datasets: Managing dynamic datasets where elements need to be added or removed.

- Sorted Data Operations: Situations where sorted data needs to be maintained efficiently.

How to Implement Insert in BST?

```
public TreeNode insertBST(TreeNode root, int val) {
   if (root == null) return new TreeNode(val);

   if (val < root.val) {
      root.left = insertBST(root.left, val);
   } else if (val > root.val) {
      root.right = insertBST(root.right, val);
```

```
    }

    return root;
}
```

How to Implement Delete in BST?

```
public TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val) {
        root.left = deleteNode(root.left, key);
    } else if (key > root.val) {
        root.right = deleteNode(root.right, key);
    } else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        TreeNode minNode = getMin(root.right);
        root.val = minNode.val;
        root.right = deleteNode(root.right, minNode.val);
    }
    return root;
}

private TreeNode getMin(TreeNode node) {
    while (node.left != null) node = node.left;
    return node;
}
```

Heaps

Why Use a Heap?

Priority-Based Access: Heaps are specialized tree-based data structures that satisfy the heap property (min-heap or max-heap), allowing efficient access and removal of the minimum or maximum element.

Use Case: Ideal for priority queues, scheduling tasks based on priority, and algorithms that require frequent extraction of extrema, such as Dijkstra's shortest path or heap sort.

Which Problems Can Be Solved Using a Heap?

- **Finding the kth largest/smallest element in an array or stream.**
- **Merging k sorted lists or arrays efficiently.**

Graphs

Why Use Graphs?

Modeling Relationships: Graphs are versatile data structures used to represent networks of nodes (vertices) connected by edges, which can be directed or undirected, weighted or unweighted.

Use Case: Essential for modeling real-world scenarios like social networks, transportation systems, web pages (for search engines), and dependency graphs in software.

Which Problems Can Be Solved Using Graphs?

- Pathfinding and connectivity: Finding shortest paths, detecting cycles, or checking if components are connected.
- Network analysis: Problems involving flow, ranking, or traversal in networks.
- Recommendation systems: Modeling user-item interactions or friendships.

Breadth-First Search (BFS)

Why Use BFS? Level-by-Level Traversal: BFS explores all nodes at the current depth before moving deeper, making it suitable for finding the shortest path in unweighted graphs. Which Problems Can Be Solved Using BFS?

- Shortest path in unweighted graphs.
- Finding connected components or levels in a graph.
- Puzzle solving, like finding the minimum moves in a game.

How to Implement BFS?

```
public class GraphBFS {

    // Adjacency list representation
     private Map<Integer, List<integer>> adjList = new HashMap&#x3C;>();</integer>
```

```
    public void addEdge(int u, int v) {
            adjList.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
            adjList.computeIfAbsent(v, k -> new ArrayList<>()).add(u); // Undirected
    }

    public void bfs(int start) {

            Queue<integer> queue = new LinkedList&#x3C;>();
            Set<integer> visited = new HashSet&#x3C;>();
           queue.add(start);
           visited.add(start);</integer></integer>

           while (!queue.isEmpty()) {
                   int node = queue.poll();
                   System.out.print(node + " ");
                   for (int neighbor : adjList.getOrDefault(node, new ArrayList<>())) {
                      if (!visited.contains(neighbor)) {
                            visited.add(neighbor);
                            queue.add(neighbor);
                       }
                   }
           }
    }

}
```

Depth-First Search (DFS)

Why Use DFS?

Deep Exploration: DFS goes as deep as possible along each branch before backtracking, useful for exhaustive searches or topological sorting.

Which Problems Can Be Solved Using DFS?

- Detecting cycles in graphs.
- Topological sorting for directed acyclic graphs (DAGs).
- Maze solving or pathfinding where all paths need exploration. How to Implement DFS?

```
public class GraphDFS {
```

```
// Adjacency list representation

private Map<Integer, List<integer>> adjList = new HashMap&#x3C;>();</integer>
public void addEdge(int u, int v) {
        adjList.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        adjList.computeIfAbsent(v, k -> new ArrayList<>()).add(u); // Undirected
}

public void dfs(int start) {
        Set<integer> visited = new HashSet&#x3C;>();
        dfsHelper(start, visited);
}

private void dfsHelper(int node, Set<integer> visited) {
          visited.add(node);
       System.out.print(node + " ");</integer>
       for (int neighbor : adjList.getOrDefault(node, new ArrayList<>())) {
              if (!visited.contains(neighbor)) {
                   dfsHelper(neighbor, visited);
              }

       }

}

}
```

Dijkstra's Algorithm

Why Use Dijkstra's Algorithm?

Shortest Path in Weighted Graphs: It finds the shortest path from a source to all nodes in a graph with non-negative weights using a priority queue.

Which Problems Can Be Solved Using Dijkstra's Algorithm?

- **Navigation systems:** Finding the shortest route in maps.
- **Network routing:** Optimizing data packet paths in computer networks.
- **Resource allocation:** Minimizing costs in weighted graphs. How to Implement Dijkstra's Algorithm?

```
public class Dijkstra {
```

```
    public static class Edge {
        int target, weight;
        Edge(int target, int weight) {
            this.target = target;
            this.weight = weight;
        }

    }

private Map<Integer, List<edge>> adjList = new HashMap<>();
        public void addEdge(int u, int v, int weight) {
        adjList.computeIfAbsent(u, k -> new ArrayList<>()).add(new Edge(v, weight));
}

public Map<Integer, Integer> dijkstra(int start) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        Map<Integer, Integer> dist = new HashMap<>();
        for (int node : adjList.keySet()) dist.put(node, Integer.MAX_VALUE);
            dist.put(start, 0);
            pq.add(new int[]{start, 0});
            while (!pq.isEmpty()) {
                int[] curr = pq.poll();
                int node = curr[0], d = curr[1];
                if (d > dist.get(node)) continue;
                for (Edge edge : adjList.getOrDefault(node, new ArrayList<>())) {

                    int newDist = d + edge.weight;
                    if (newDist < dist.get(edge.target)) {
                        dist.put(edge.target, newDist);
                        pq.add(new int[]{edge.target, newDist});

                     }
        }

}

return dist;

}

}
```

Dynamic Programming

Why Use Dynamic Programming?

Optimal Substructure and Overlapping Subproblems: DP breaks down complex problems into simpler subproblems, storing results to avoid redundant computations, leading to efficient solutions for optimization problems.

Use Case: Perfect for problems where the solution depends on optimal solutions to smaller instances, such as resource allocation, sequencing, or pathfinding with constraints.

Which Problems Can Be Solved Using Dynamic Programming?

- Classic optimization: Knapsack problem, longest common subsequence, matrix chain multiplication.
- Sequence problems: Fibonacci sequence, edit distance, longest increasing subsequence.
- Graph problems: Shortest paths with constraints (e.g., Bellman-Ford) or all-pairs shortest paths (Floyd-Warshall). How to Implement Dynamic Programming? Step 1: Memoization (Top-Down Approach) Use recursion with a cache to store results of subproblems. Example: Fibonacci sequence.

```
public static int fibMemo(int n, int[] memo) {

    if (n <= 1) return n;

    if (memo[n] != -1) return memo[n];
        memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
        return memo[n];
    }

int[] memo = new int[n + 1];
Arrays.fill(memo, -1);
fibMemo(n, memo);
```

Step 2: Tabulation (Bottom-Up Approach)

Build a table iteratively from base cases to the target.

Example: 0/1 Knapsack.

```
public static int knapsack(int[] weights, int[] values, int capacity, int n) {
        int[][] dp = new int[n + 1][capacity + 1];
        for (int i = 0; i <= n; i++) {
             for (int w = 0; w <= capacity; w++) {
                if (i == 0 || w == 0) {
                       dp[i][w] = 0;
                } else if (weights[i - 1] <= w) {
                  dp[i][w] = Math.max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
                 } else {
                     dp[i][w] = dp[i - 1][w];

                 }

        }

}

return dp[n][capacity];

}
```